

파일 입출력

텍스트와 바이너리

1부 복습: read, write, lseek

2부 학습: fread, fwrite, fseek

수업 구성

1부 · 텍스트 저장과 읽기: read, write, lseek

2부 · 바이너리 저장과 읽기: fread, fwrite, fseek

두 함수군 모두 텍스트와 바이너리를 처리한다.

예제의 저장 형식과 API의 차이를 구분한다.

1부 · 복습

텍스트 저장과 읽기

`read()`, `write()`, `lseek()`로 파일의 바이트와 위치를 확인합니다.

텍스트 파일의 바이트

예제는 ASCII 문자열 ABCDEFGHIJ다.

파일에는 문자 10개의 바이트를 저장한다.

C 문자열 끝의 NUL은 이번 파일에 쓰지 않는다.

파일 위치는 다음에 읽거나 쓸 바이트의 위치다.

open과 close 복습

```
int fd = open("text.txt",
    O_RDWR | O_CREAT | O_TRUNC, 0644);
if (fd == -1) {
    perror("open");
    return 1;
}
/* write, read, lseek 실행 */
if (close(fd) == -1) return 1;
```

핵심

open은 성공하면 0 이상의 FD를 반환한다.

동작

O_TRUNC는 기존 파일 내용을 지운다.

확인

사용이 끝난 FD는 close로 닫는다.

write · 텍스트 저장

```
ssize_t write(int fd, const void *buf,  
              size_t count);  
  
const char text[] = "ABCDEFGHJIJ";  
ssize_t n = write(fd, text, sizeof text - 1);  
/* 요청은 10바이트 */  
/* 성공: 0 이상, 오류: -1 */
```

핵심

buf에서 count바이트를 파일에 쓴다.

동작

반환값은 실제로 쓴 바이트 수다.

확인

부분 쓰기이면 남은 바이트를 다시 쓴다.

read · 텍스트 읽기

```
char buf[16];
ssize_t n = read(fd, buf, sizeof buf - 1);
if (n == -1) {
    /* 오류 처리 */
} else {
    buf[n] = '\0';
    printf("%s", buf);
}
```

핵심

최대 count바이트를 버퍼에 저장한다.

동작

양수는 읽은 길이, 0은 EOF다.

확인

문자열로 쓸 때 NUL 공간을 남긴다.

lseek · 읽기와 쓰기 위치

```
off_t lseek(int fd, off_t offset, int whence);
```

```
lseek(fd, 0, SEEK_SET); /* 파일 시작 */  
lseek(fd, 3, SEEK_SET); /* 시작에서 3 */  
lseek(fd, -2, SEEK_CUR); /* 현재에서 -2 */  
lseek(fd, 0, SEEK_END); /* 파일 끝 */
```

핵심

offset은 기준점에서 이동할 바이트 수다.

동작

성공하면 새 파일 위치를 반환한다.

확인

실패하면 (off_t)-1을 반환한다.

텍스트 위치 예제

```
/* text.txt: ABCDEFGHIJ, 0_RDWR */  
lseek(fd, 3, SEEK_SET);  
read(fd, buf, 2);  
/* 읽은 데이터: DE */  
  
lseek(fd, 3, SEEK_SET);  
write(fd, "XY", 2);  
/* 파일 결과: ABCXYFGHIJ */
```

핵심

시작에서 3바이트 이동하면 D 위치다.

동작

2바이트를 읽으면 DE, 다음 위치는 5다.

확인

다시 위치 3에 XY를 덮어쓴다.

1부 확인 · 반환값과 버퍼

write가 4를 반환하면 4바이트만 처리했다.

read가 2를 반환하면 읽은 2바이트를 사용한다.

char buf[4]의 유효 인덱스는 0~3이다.

끝 밖 쓰기는 overflow, 시작 앞 쓰기는 underwrite다.

데이터 부족을 뜻하는 underflow와 구별한다.

2부

바이너리 저장과 읽기

fread(), fwrite(), fseek()로 정수 배열을 저장하고 읽습니다.

텍스트 표현과 바이너리 표현

텍스트: 숫자 1234를 문자 "1234"로 저장

바이너리: 정수 값의 메모리 바이트를 저장

텍스트는 문자 해석과 숫자 변환이 필요하다.

바이너리는 자료형과 저장 형식을 알아야 한다.

이번 예제는 같은 환경에서 int 배열을 읽고 쓴다.

FILE*와 fopen

```
FILE *fp = fopen("numbers.bin", "wb");
if (fp == NULL) {
    perror("fopen");
    return 1;
}
/* fwrite로 저장 */
if (fclose(fp) == EOF) return 1;
```

핵심

FILE*는 C 라이브러리의 스트림이다.

동작

wb는 새로 쓰기, rb는 읽기 모드다.

확인

fopen 실패는 NULL로 확인한다.

fwrite · 바이너리 저장

```
/* fwrite(ptr, size, nmemb, stream) */
int values[] = {10, 20, 30};

size_t n = fwrite(values,
                   sizeof values[0], 3, fp);
if (n != 3) {
    /* 쓰기 오류 처리 */
}
/* 요청 크기: 3 * sizeof(int) */
```

핵심

size는 항목 하나의 크기다.

동작

nmemb는 저장할 항목 개수다.

확인

반환값은 성공적으로 쓴 항목 수다.

fread · 바이너리 읽기

```
/* fp: numbers.bin을 rb로 연 스트림 */
int values[3];
size_t n = fread(values,
                  sizeof values[0], 3, fp);
if (n != 3) {
    if (ferror(fp)) { /* 읽기 오류 */ }
    else { /* 파일 내용이 부족함 */ }
}
/* n == 3일 때 값: 10, 20, 30 */
```

핵심

같은 항목 크기와 개수로 읽는다.

동작

n이 3이면 int 3개를 완전히 읽었다.

확인

부족하면 EOF와 오류를 구별한다.

항목 수와 바이트 수

`fread(values, sizeof(int), 3, fp)`의 단위는 `int`다.

`fread(buf, 1, sizeof buf, fp)`의 단위는 바이트다.

요청 바이트 수는 `size × nmemb`다.

요청 전체를 담을 버퍼 공간이 필요하다.

`fread`는 문자열 끝의 `NUL`을 붙이지 않는다.

fseek · 바이너리 파일 위치

```
int fseek(FILE *stream, long offset,
          int whence);

/* 두 번째 int: 인덱스 1 */
long offset = (long)sizeof(int);
if (fseek(fp, offset, SEEK_SET) != 0) {
    /* 위치 이동 오류 처리 */
}
```

핵심

offset은 항목 개수가 아닌 바이트 수다.

동작

SEEK_SET, SEEK_CUR, SEEK_END를 사용한다.

확인

성공은 0, 실패는 0이 아닌 값이다.

원하는 항목 하나 읽기

```
/* fp: numbers.bin을 rb로 연 스트림 */
int value;
long offset = (long)sizeof value;
if (fseek(fp, offset, SEEK_SET) != 0)
    goto fail;
if (fread(&value, sizeof value, 1, fp) != 1)
    goto fail;
printf("%d\n", value); /* 20 */
```

핵심

인덱스 k 의 위치는 $k \times$ 항목 크기다.

동작

두 번째 값은 인덱스 1의 20이다.

확인

fread 반환값 1을 확인한 뒤 사용한다.

원하는 항목 하나 수정

```
FILE *fp = fopen("numbers.bin", "r+b");
if (!fp) return 1;
int value = 99;
if (fseek(fp, (long)sizeof value, SEEK_SET) != 0)
    goto fail;
if (fwrite(&value, sizeof value, 1, fp) != 1)
    goto fail;
if (fclose(fp) == EOF) return 1;
```

핵심

기존 파일을 r+b 모드로 연다.

동작

두 번째 int 위치에 99를 덮어쓴다.

확인

수정 후 배열은 10, 99, 30이다.

읽기와 쓰기 전환

같은 FILE*로 읽고 쓸 때 전환 규칙이 있다.

쓰기 후 읽기: fflush 또는 위치 지정이 필요하다.

읽기 후 쓰기: 보통 위치 지정이 필요하다.

이 수업에서는 fseek로 다음 위치를 지정한다.

fseek 성공 여부를 확인한 뒤 다음 작업을 한다.

버퍼링과 파일 닫기

`stdio`는 라이브러리 버퍼로 입출력을 모을 수 있다.

`fwrite` 성공 뒤에도 출력 버퍼에 남을 수 있다.

`fflush`는 출력 버퍼의 데이터를 커널로 내보낸다.

`fclose`는 출력 버퍼를 처리하고 파일을 닫는다.

두 함수 모두 성공 0, 실패 EOF를 검사한다.

실습 · 저장, 선택 읽기, 수정

1. int 배열 {10, 20, 30}을 fwrite로 저장한다.
2. rb로 열고 fread로 전체 배열을 확인한다.
3. fseek로 두 번째 항목을 읽어 20을 확인한다.
4. r+b로 열어 두 번째 항목을 99로 바꾼다.
5. 다시 읽어 {10, 99, 30}인지 확인한다.

두 함수군의 대응

파일 열기와 닫기: `open / close`, `fopen / fclose`

읽기와 쓰기: `read / write`, `fread / fwrite`

위치 지정: `lseek`는 새 위치, `fseek`는 성공 여부 반환

파일 표현: 정수 `FD`, `FILE*` 스트림

저장 형식은 프로그램이 기록하는 바이트로 결정한다.

정리 퀴즈

1. write에 정수 주소를 전달하면 무엇을 저장하는가?
2. int 3개를 쓴 fwrite의 성공 반환값은?
3. 세 번째 int의 시작 위치는 몇 바이트인가?
4. fseek 성공 시 반환값과 현재 위치는 같은가?